1. Introduction to Multimodal Prompting

- **Single Modality Prompting:** This is when you provide only one type of input to a Large Language Model (LLM), typically text, and receive a text output.
- **Multimodal Prompting:** This occurs when you provide the LLM with more than one type of input simultaneously. For example, you might upload a document alongside a text prompt, or provide an image along with text-based questions about that image.
-

2. Why Modern LLMs Excel at Multimodal Tasks

Current waves of LLMs are highly powerful at processing images and documents due to three primary advancements:

A. Computer Vision (CV) Capabilities Over OCR

- **The Problem with Older LLMs:** They relied on Optical Character Recognition (OCR), which only focuses on extracting textual data. If a document contained graphs or tables, OCR would strip out the layout and just output raw, jumbled text (like standalone numbers or letters), feeding "garbage" to the LLM and resulting in poor outputs.
- **The Power of Computer Vision:** Modern LLMs use CV to treat uploaded documents and files as images. This means the model reads every pixel, which maintains **layout awareness**. The LLM can understand the spatial relationship between elements, recognizing that specific text belongs to a specific table column or graph, leading to highly accurate information retention.

B. Large Context Windows

- **What it is:** The "context window" refers to the maximum length of input (measured in tokens) an LLM can process at one time.
- **The Problem with Older LLMs:** They had small context windows (typically 10k to 20k tokens), making it impossible to upload large files like a 100-page annual report or maintain long, multi-turn conversations without the model forgetting earlier inputs and crashing.
- **Modern Advancements:** Current LLMs boast context windows of up to 1 million tokens, which is roughly equivalent to a 1,300 to 1,500-page book. This allows users to easily upload massive documents and engage in extended conversations.

C. Cache Augmented Generation

- **The Problem with Older Methods:** Previously, answering multiple questions from a document was like an "open book test" for the LLM; it had to re-read and re-process the entire document for every single question, causing high latency and computational costs.
- **How Cache Works:** Modern LLMs read and process the uploaded document just once, storing their understanding in a cache memory. Whenever you ask subsequent questions, the LLM retrieves the answers directly from the cache, drastically speeding up response times.

3. Best Practices for Structuring Prompts

Even though modern LLMs are powerful enough to give decent answers (e.g., a 70/100 score) with simple prompts, following a structured framework can elevate the quality of the output significantly (to a 90+/100 score). You should avoid simply uploading a document and asking a direct question. Instead, divide your prompt into three primary components using delimiter tags:

A. Role (Persona)

- Define exactly who the LLM is supposed to be (e.g., a Senior Financial Analyst or a Senior Data Scientist).
- The persona dictates the vocabulary, depth of explanation, and the types of examples the LLM will use.
- **Anti-Persona Elements:** You can also tell the model what *not* to be, such as instructing a Senior Data Scientist persona to "avoid jargon and only use real-world analogies".

B. Context

- State what file or document you have uploaded.
- **Target Prompting:** If needed, instruct the LLM to only focus on a specific section (e.g., "Section 3") of a massive document to give it a predefined reading lens.
- **Context Grounding:** This is crucial. Explicitly instruct the LLM to base its output *strictly* on the provided document and not on its pre-trained, external knowledge. Furthermore, tell the LLM to clearly state "No" or "Information not found" if the document does not contain the answer. This prevents the LLM from hallucinating or drifting off-topic during long conversations.

C. Task

- Clearly define the query or objective you want the LLM to achieve.

4. The "Chain of Verification" (CoVe) Technique

To further reduce hallucinations and increase output authenticity, embed the **Chain of Verification** framework into the "Task" section of your prompt. This is a 4-step method where the LLM acts as its own critic:

1. **Initial Draft:** Ask the LLM to write a quick, rough draft answering your query based on the document.
2. **Generate Verification Questions:** Instruct the LLM to look at its own draft and generate specific questions (e.g., 3 questions) to fact-check the claims it just made.
3. **Answer the Verification Questions:** The LLM must then search the uploaded document to answer those verification questions, preferably extracting exact quotes to verify the facts.
4. **Final Revised Output:** The LLM generates a final answer based on the initial draft, replacing any hallucinated facts with the corrected, verified information found in Step 3.

Real-World Example Provided in the Lecture: When processing a 102-page Accenture Annual Report to find the "Top 3 Revenue Drivers":

- A basic prompt just copy-pasted generic revenue tables from the document without actually answering the specific question.
- Applying the structured template with **Role, Context Grounding, and CoVe** allowed the LLM to successfully isolate the actual top three specific drivers (Americas region, Products industry group, and Consulting work) and provided exact quotes from the text to prove its accuracy